

fn zcdp_delta

Michael Shoemate (@Shoeboxam)

August 2, 2026

Proves soundness of `fn zcdp_delta`. This proof is an adaptation of [subsection 2.3](#) of [\[CKS20\]](#).

1 Bound Derivation

Definition 1.1. (Privacy Loss Random Variable). Let $M : \mathcal{X}^n \rightarrow \mathbb{Y}$ be a randomized algorithm. Let $x, x' \in \mathcal{X}^n$ be neighboring inputs. Define $f : \mathcal{Y} \rightarrow \mathbb{R}$ by $f(y) = \log \left(\frac{\mathbb{P}[M(x)=y]}{\mathbb{P}[M(x')=y]} \right)$. Let $Z = f(M(x))$, the privacy loss random variable, denoted $Z \leftarrow \text{PrivLoss}(M(x)||M(x'))$.

Lemma 1.2. [\[CKS20\]](#) Let $\epsilon, \delta \geq 0$. Let $M : \mathcal{X}^n \rightarrow \mathcal{Y}$ be a randomized algorithm. Then M satisfies (ϵ, δ) -differential privacy if and only if

$$\delta \geq \mathbb{E}_{Z \leftarrow \text{PrivLoss}(M(x)||M(x'))} [\max(0, 1 - e^{\epsilon - Z})] \quad (1)$$

(2)

for all $x, x' \in \mathcal{X}^n$ differing on a single element.

Proof. Fix neighboring inputs $x, x' \in \mathcal{X}^n$. Let $f : \mathcal{Y} \rightarrow \mathbb{R}$ be as in [1.1](#). For notational simplicity, let $Y = M(x)$, $Y' = M(x')$, $Z = f(Y)$ and $Z' = -f(Y')$. This is equivalent to $Z \leftarrow \text{PrivLoss}(M(x)||M(x'))$. Our first goal is to prove that

$$\sup_{E \subset \mathcal{Y}} \mathbb{P}[Y \in E] - e^\epsilon \mathbb{P}[Y' \in E] = \mathbb{E}[\max\{0, 1 - e^{\epsilon - Z}\}]. \quad (3)$$

For any $E \subset \mathcal{Y}$, we have

$$\mathbb{P}[Y' \in E] = \mathbb{E}[\mathbb{I}[Y' \in E]] = \mathbb{E}[\mathbb{I}[Y \in E] e^{-f(Y)}]. \quad (4)$$

This is because $e^{-f(y)} = \frac{\mathbb{P}[Y'=y]}{\mathbb{P}[Y=y]}$.

Thus, for all $E \subset \mathcal{Y}$, we have

$$\mathbb{P}[Y \in E] - e^\epsilon \mathbb{P}[Y' \in E] = \mathbb{E} \left[\mathbb{I}[Y \in E] (1 - e^{\epsilon - f(Y)}) \right] \quad (5)$$

Now it is easy to identify the worst event as $E = \{y \in \mathcal{Y} : 1 - e^{\epsilon - f(y)} > 0\}$. Thus

$$\sup_{E \subset \mathcal{Y}} \mathbb{P}[Y \in E] - e^\epsilon \mathbb{P}[Y' \in E] = \mathbb{E} \left[\mathbb{I}[1 - e^{\epsilon - f(Y)} > 0] (1 - e^{\epsilon - f(Y)}) \right] = \mathbb{E}[\max\{0, 1 - e^{\epsilon - Z}\}] \quad (6)$$

□

Theorem 1.3. [CKS20] Let $M : \mathcal{X}^n \rightarrow \mathcal{Y}$ be a randomized algorithm. Let $\alpha \in (1, \infty)$ and $\epsilon \geq 0$. Suppose $D_\alpha(M(x)||M(x')) \leq \tau$ for all $x, x' \in \mathcal{X}^n$ differing in a single entry.¹ Then M is (ϵ, δ) -differentially private for

$$\delta = \frac{e^{(\alpha-1)(\tau-\epsilon)}}{\alpha-1} \left(1 - \frac{1}{\alpha}\right)^\alpha \quad (7)$$

Proof. Fix neighboring $x, x' \in \mathcal{X}^n$ and let $Z \leftarrow \text{PrivLoss}(M(x)||M(x'))$. We have

$$\mathbb{E}[e^{(\alpha-1)Z}] = e^{(\alpha-1)D_\alpha(M(x)||M(x'))} \leq e^{(\alpha-1)\tau} \quad (8)$$

By 1.2, our goal is to prove that $\delta \geq \mathbb{E}[\max\{0, 1 - e^{\epsilon-Z}\}]$. Our approach is to pick $c > 0$ such that $\max\{0, 1 - e^{\epsilon-Z}\} \leq ce^{(\alpha-1)z}$ for all $z \in \mathbb{R}$. Then

$$\mathbb{E}[\max\{0, 1 - e^{\epsilon-Z}\}] \leq \mathbb{E}[ce^{(\alpha-1)z}] \leq ce^{(\alpha-1)\tau}. \quad (9)$$

We identify the smallest possible value of c :

$$c = \sup_{z \in \mathbb{R}} \frac{\max\{0, 1 - e^{\epsilon-z}\}}{e^{(\alpha-1)z}} = \sup_{z \in \mathbb{R}} e^{z-\alpha z} - e^{\epsilon-\alpha z} = \sup_{z \in \mathbb{R}} f(z) \quad (10)$$

where $f(z) = e^{z-\alpha z} - e^{\epsilon-\alpha z}$. We have

$$f'(z) = e^{z-\alpha z}(1-\alpha) - e^{\epsilon-\alpha z}(-\alpha) = e^{-\alpha z}(\alpha e^\epsilon - (\alpha-1)e^z) \quad (11)$$

Clearly $f'(z) = 0 \iff e^z = \frac{\alpha}{\alpha-1}e^\epsilon \iff z = \epsilon - \log(1-1/\alpha)$. Thus

$$c = f(\epsilon - \log(1-1/\alpha)) \quad (12)$$

$$= \left(\frac{\alpha}{\alpha-1}e^\epsilon\right)^{1-\alpha} - e^\epsilon \left(\frac{\alpha}{\alpha-1}e^\epsilon\right)^{-\alpha} \quad (13)$$

$$= \left(\frac{\alpha}{\alpha-1}e^\epsilon - e^\epsilon\right) \left(\frac{\alpha}{\alpha-1}e^{-\epsilon}\right)^\alpha \quad (14)$$

$$= \frac{e^\epsilon}{\alpha-1} \left(1 - \frac{1}{\alpha}\right)^\alpha e^{-\alpha\epsilon}. \quad (15)$$

Thus

$$\mathbb{E}[\max\{0, 1 - e^{\epsilon-Z}\}] \leq \frac{e^\epsilon}{\alpha-1} \left(1 - \frac{1}{\alpha}\right)^\alpha e^{-\alpha\epsilon} e^{(\alpha-1)\tau} = \frac{e^{(\alpha-1)(\tau-\epsilon)}}{\alpha-1} \left(1 - \frac{1}{\alpha}\right)^\alpha = \delta \quad (16)$$

□

Theorem 1.4. [ALC⁺21, Lemma 1, Equation (25)] Let M satisfy (α, γ) -RDP for $\alpha > 1$. If $\epsilon > \gamma$, then M is (ϵ, δ) -differentially private for

$$\delta = \frac{e^{(\alpha-1)\gamma} - 1}{\alpha(e^{(\alpha-1)\epsilon} - 1)}. \quad (17)$$

Proof. The second term of [ALC⁺21, Lemma 1, Equation (25)] states that, when $0 < \alpha\delta < 1$, the smallest guaranteed approximate-DP epsilon is at most

$$\frac{1}{\alpha-1} \log \left(\frac{e^{(\alpha-1)\gamma} - 1}{\alpha\delta} + 1 \right). \quad (18)$$

Substitution of the stated δ makes this expression equal to ϵ . Moreover, $\epsilon > \gamma$ implies $e^{(\alpha-1)\epsilon} - 1 > e^{(\alpha-1)\gamma} - 1$, and therefore $0 < \alpha\delta < 1$, satisfying the lemma's condition. □

¹This is the definition of (α, τ) -Rényi differential privacy.

Corollary 1. [CKS20] Let $M : \mathcal{X}^n \rightarrow \mathcal{Y}$ be a randomized algorithm. Let $\alpha \in (1, \infty)$ and $\epsilon \geq 0$. Suppose $D_\alpha(M(x)||M(x')) \leq \tau$ for all $x, x' \in \mathcal{X}^n$ differing in a single entry. Then M is (ϵ, δ) -differentially private for

$$\epsilon = \tau + \frac{\ln(1/\delta) + (\alpha - 1) \ln(1 - 1/\alpha) - \ln(\alpha)}{\alpha - 1} \quad (19)$$

Proof. This follows by rearranging 1.3. □

Corollary 2. Let $M : \mathcal{X}^n \rightarrow \mathcal{Y}$ be a randomized algorithm satisfying ρ -concentrated differential privacy. Then M is (ϵ, δ) -differentially private for any $0 < \delta \leq 1$ and

$$\epsilon = \inf_{\alpha \in (1, \infty)} \alpha \rho + \frac{\ln(1/\delta) + (\alpha - 1) \ln(1 - 1/\alpha) - \ln(\alpha)}{\alpha - 1} \quad (20)$$

Proof. This follows from 1 by taking the infimum over all divergence parameters α . □

2 Hoare Triple

Precondition

Compiler-verified

None

Caller-verified

None

Pseudocode

```

1 def zcdp_delta(rho: float, epsilon: float) -> float:
2     validate_nonnegative_finite_or_infinite(rho, epsilon)
3     if rho == 0.0 or epsilon == infinity:
4         return 0.0
5     if rho == infinity:
6         return 1.0
7
8     # Ordinary floating-point optimization only selects candidate orders.
9     # Every accepted candidate gives a valid bound.
10    alpha_lower = 1.0 + sqrt(machine_epsilon)
11    alpha_upper = choose_finite_search_bound(rho, epsilon)
12    cks_bracket = minimize_in_log_domain(
13        lambda alpha: approximate_cks_log_delta(alpha, rho, epsilon),
14        alpha_lower,
15        alpha_upper,
16    )
17
18    # Re-evaluate candidates with outward-rounded interval arithmetic.
19    log_delta_upper = infinity
20    for alpha in candidates(cks_bracket, alpha_lower, alpha_upper):
21        alpha = interval_point(alpha)
22        rho_interval = interval_point(rho)
23        epsilon_interval = interval_point(epsilon)
24        log_bound = (
25            (alpha - 1) * (alpha * rho_interval - epsilon_interval)
26            + alpha * log(1 - 1 / alpha)
27            - log(alpha - 1)
28        )
29        log_delta_upper = min(log_delta_upper, log_bound.upper)

```

```

30
31 # Asoodeh et al., Lemma 1, Equation (25), supplies a second bound
32 # whenever epsilon > alpha * rho.
33 if epsilon > rho:
34     asoodeh_upper = interior_upper_bound(epsilon / rho)
35     asoodeh_bracket = minimize_in_log_domain(
36         lambda alpha: approximate_asoodeh_log_delta(
37             alpha, rho, epsilon
38         ),
39         alpha_lower,
40         asoodeh_upper,
41     )
42     for alpha in candidates(asoodeh_bracket):
43         if epsilon <= alpha * rho:
44             continue
45         alpha = interval_point(alpha)
46         x = (alpha - 1) * alpha * interval_point(rho)
47         y = (alpha - 1) * interval_point(epsilon)
48         log_bound = log_expml(x) - log(alpha) - log_expml(y)
49         log_delta_upper = min(log_delta_upper, log_bound.upper)
50
51     return conservative_exp_clamped_to_unit_interval(log_delta_upper)

```

Postcondition

Theorem 2.1. For any possible setting of ρ and ϵ , `zcdp_delta` either returns an error, or a δ such that any ρ -differentially private measurement is also (ϵ, δ) -differentially private.

Proof. The validation and special-case branches return an error or a trivial valid bound. Otherwise, ordinary floating-point optimization only selects a finite set A of candidate orders strictly greater than one. Its accuracy affects tightness but not soundness.

For each candidate, the code evaluates the logarithm of either the bound from 1.3, or, when $\epsilon > \alpha\rho$, the bound from 1.4, with the zCDP guarantee $\gamma = \alpha\rho$. These evaluations use `DInterval`, whose operations round outwards, so each upper endpoint is at least the exact logarithm of the corresponding valid bound. The minimum remains valid because every candidate bounds the same privacy quantity from above.

Finally, `InfExp` rounds exponentiation upwards and returns the smallest positive `f64` on underflow. Any interval-evaluation failure contributes the trivial bound one. Thus the returned `delta` satisfies the postcondition. $\square$

References

- [ALC⁺21] Shahab Asoodeh, Jiachun Liao, Flavio P. Calmon, Oliver Kosut, and Lalitha Sankar. Three variants of differential privacy: Lossless conversion and applications. *IEEE Journal on Selected Areas in Information Theory*, 2(1):208–222, 2021.
- [CKS20] Clément L. Canonne, Gautam Kamath, and Thomas Steinke. The discrete gaussian for differential privacy. *CoRR*, abs/2004.00010, 2020.